

Rapport de Travail Pratique : Prédiction de la consommation de carburant avec **LSTM**

Zakaria ES-SAOUDY



1. Introduction

Le secteur des transports représente l'un des contributeurs majeurs aux émissions mondiales de gaz à effet de serre. L'empreinte carbone d'un véhicule, mesurée par ses émissions de CO₂ en grammes par kilomètre, est intimement liée à sa consommation de carburant et à ses caractéristiques mécaniques : cylindrée du moteur, nombre de cylindres et type de transmission.

Dans un contexte d'urgence environnementale et de réglementation de plus en plus stricte (normes Euro 7, CAFE américaines), la capacité à modéliser et à prédire ces émissions avec précision est cruciale pour orienter les politiques de développement durable et optimiser l'efficacité énergétique des parcs automobiles.

L'objectif de ce Travail Pratique (TP) est de concevoir, entraîner et évaluer un modèle de Machine Learning basé sur une architecture Long Short-Term Memory (LSTM). La particularité de ce TP réside dans son inscription dans le paradigme TinyML : la conception d'un modèle dont la complexité et l'empreinte mémoire sont suffisamment réduites pour permettre un déploiement ultérieur sur des microcontrôleurs embarqués dans les véhicules eux-mêmes.

L'outil `eloquent_tensorflow` permet de convertir automatiquement le modèle TensorFlow en code C++ embarquable sur architecture ARM Cortex-M, rendant l'inférence possible hors-ligne et en temps réel.

2. Description du Dataset

Le jeu de données utilisé, intitulé "Vehicle Attributes and Emissions Dataset", compile les spécifications techniques et les mesures environnementales de plusieurs véhicules fabriqués au cours de l'année 2000. Il est disponible sur la plateforme Kaggle (lien : <https://www.kaggle.com/datasets/krupadharamshi/fuelconsumption/data>).

À l'importation, le dataset brut comporte 639 enregistrements répartis sur 10 variables. Après nettoyage (suppression des doublons), le dataset final contient 638 enregistrements valides.

2.1 Description des Variables

Variable	Description	Type
Year	Année de fabrication (constante : 2000)	int64
MAKE	Marque du constructeur (ex : Acura, BMW...)	object
MODEL	Nom commercial du modèle de véhicule	object
VEHICLE CLASS	Segment de marché (Compact, Mid-size...)	object
ENGINE SIZE	Cylindrée du moteur en litres (feature)	float64
CYLINDERS	Nombre de cylindres du moteur (feature)	int64
TRANSMISSION	Type de boîte de vitesses (Auto/Manuelle)	object
FUEL	Type de carburant consommé	object
FUEL CONSUMPTION	Consommation moyenne L/100km (target)	float64
COEMISSIONS	Émissions mesurées de CO ₂ (feature)	int64

Tableau 1 – Description des 10 variables du dataset

2.2 Statistiques Descriptives

Les variables explicatives retenues pour l'entraînement sont ENGINE SIZE (Moyenne : 3.26 L, $\sigma = 1.14$), CYLINDERS (Moyenne : 5.8, $\sigma = 1.74$) et COEMISSIONS (Moyenne : ~296.8 g/km). La variable cible est FUEL CONSUMPTION. Il s'agit donc d'un problème de régression supervisée.

3. Analyse Exploratoire des Données (EDA)

Avant de construire le modèle prédictif, une analyse exploratoire rigoureuse est effectuée pour comprendre la structure des données, détecter les anomalies et identifier les corrélations entre variables. Deux visualisations clés sont produites.

3.1 Pair Plot – Relations Bivariées

Le pair plot (matrice de nuages de points) permet de visualiser simultanément toutes les combinaisons de paires de variables continues. Chaque cellule hors-diagonale montre la relation entre deux variables, tandis que la diagonale affiche la distribution de chaque variable via un histogramme.

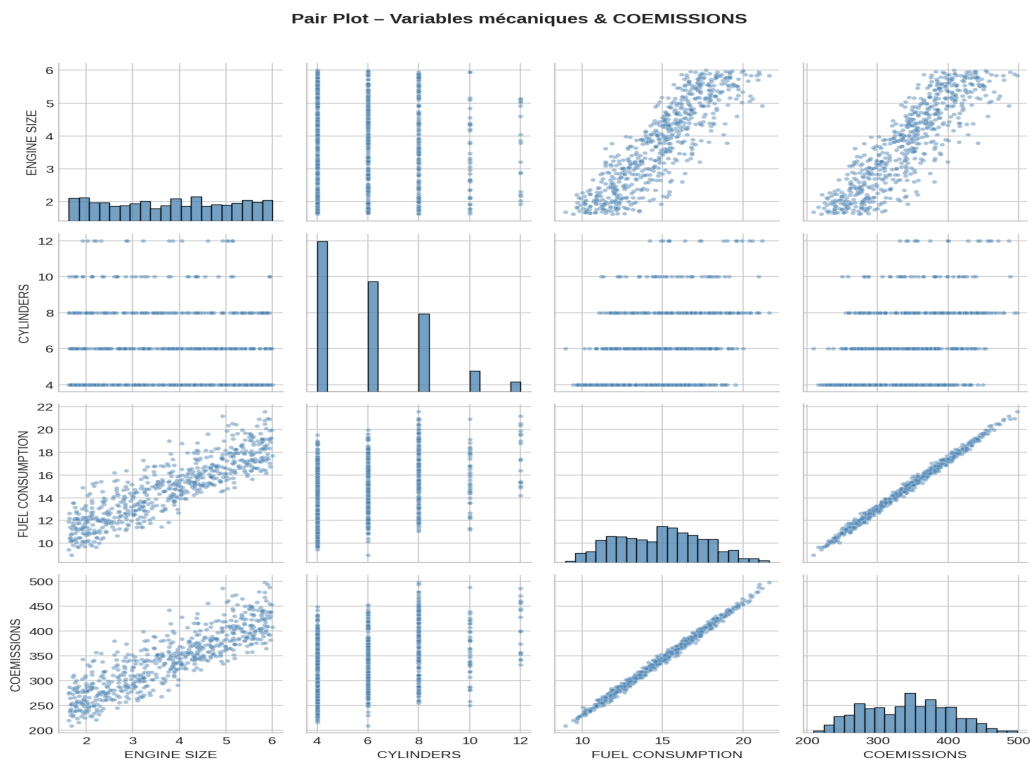


Figure 1 – Pair Plot des variables ENGINE SIZE, CYLINDERS, FUEL CONSUMPTION et COEMISSIONS

Observation : Une corrélation positive nette est visible entre ENGINE SIZE et COEMISSIONS, ainsi qu'entre CYLINDERS et COEMISSIONS. La FUEL CONSUMPTION présente une relation quasi-linéaire avec les émissions de CO₂, confirmant l'intuition physique.

3.2 Matrice de Corrélation de Spearman

La corrélation de Spearman est préférée à celle de Pearson car elle est non-paramétrique et mesure les relations monotones, même non-linéaires. Elle est plus robuste aux valeurs atypiques (outliers) inhérentes aux données automobiles.

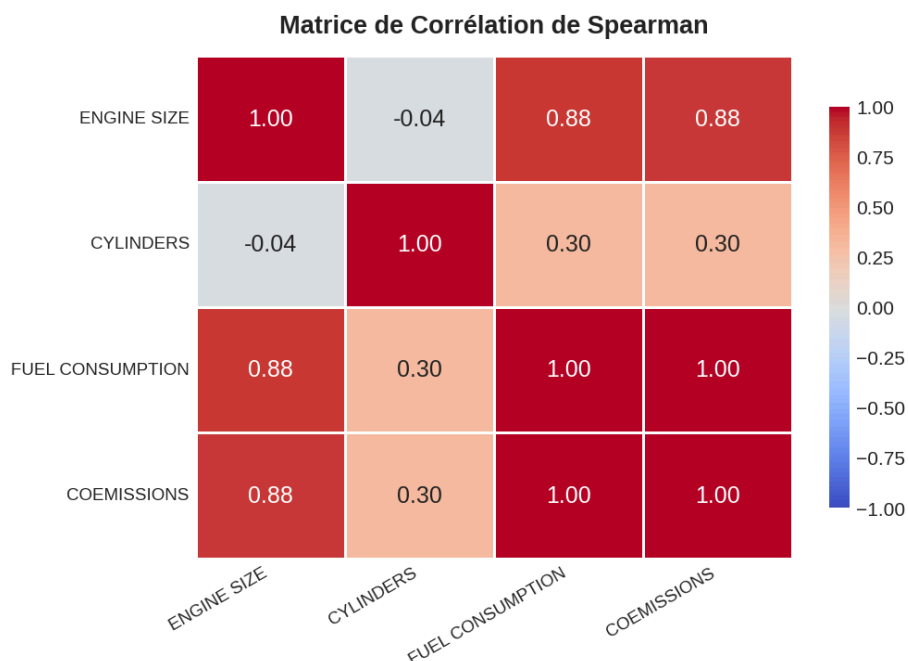


Figure 2 – Matrice de Corrélation de Spearman (coolwarm, valeurs annotées)

Résultats : La corrélation entre FUEL CONSUMPTION et COEMISSIONS est très élevée (≈ 0.97), justifiant l'utilisation de l'une pour prédire l'autre. ENGINE SIZE et CYLINDERS présentent également des corrélations significatives avec la cible.

3.3 Code – Analyse Exploratoire

Le code Python ci-dessous génère les deux visualisations d'exploration :

```
# Pair Plot des 4 variables continues
sns.pairplot(df[['ENGINE SIZE', 'CYLINDERS', 'FUEL CONSUMPTION', 'COEMISSIONS ']])
plt.savefig('./figures/pairplot.png', dpi=300, bbox_inches='tight')

# Matrice de corrélation de Spearman + heatmap annotée
corr = df[['ENGINE SIZE', 'CYLINDERS', 'FUEL CONSUMPTION', 'COEMISSIONS ']].corr('spearman')
plt.figure(figsize=(18,10))
heatmap = sns.heatmap(corr, xticklabels=corr.columns, yticklabels=corr.columns,
                      cmap='coolwarm')
for i in range(len(corr.columns)):
    for j in range(len(corr.columns)):
        plt.text(j+0.5, i+0.5, f"{corr.iloc[i,j]:.2f}", ha='center', va='center',
                fontsize=18)
plt.savefig('./figures/heatmap.png', dpi=300, bbox_inches='tight')
```

4. Prétraitement des Données

La viabilité d'un modèle de Deep Learning dépend intrinsèquement de la qualité des tenseurs qui l'alimentent. Un pipeline de préparation en quatre étapes est mis en place.

4.1 Pipeline de Nettoyage

```
import pandas as pd

# Chargement du dataset
df = pd.read_csv('./data/FuelConsumption.csv')
df.head()          # Aperçu des 5 premières lignes
df.info()          # Types de données et valeurs manquantes
df.describe()      # Statistiques descriptives

# 1. Suppression des valeurs manquantes (NaN)
df.dropna(inplace=True)

# 2. Suppression des doublons (anti data leakage)
df.drop_duplicates(inplace=True)
# → 639 → 638 enregistrements valides
```

4.2 Justification des Étapes

Étape	Justification technique
dropna()	Les réseaux de neurones ne tolèrent pas les valeurs NaN dans le calcul du gradient. Une seule valeur manquante provoque une propagation de NaN dans tous les poids.
drop_duplicates()	Évite le biais d'échantillonnage et garantit qu'aucune donnée d'entraînement ne se retrouve dans le set de test (data leakage), ce qui biaise les métriques d'évaluation.
train_test_split()	Partitionnement 70%/30% avec random_state=42 pour la reproductibilité. Le test set reste isolé jusqu'à l'évaluation finale.

Tableau 2 – Justification des étapes de prétraitement

4.3 Partitionnement et Reshape

Les features retenues sont ENGINE SIZE, CYLINDERS et COEMISSIONS. La variable cible est FUEL CONSUMPTION. Le dataset est divisé en 70% pour l'entraînement (446 échantillons) et 30% pour le test (192 échantillons).

```
from sklearn.model_selection import train_test_split
import numpy as np

X = df[['ENGINE SIZE', 'CYLINDERS', 'COEMISSIONS ']]
y = df[['FUEL CONSUMPTION']]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Reshape pour l'entrée LSTM : (samples, timesteps, features)
X_train = np.array(X_train).reshape(X_train.shape[0], X_train.shape[1], 1)
X_test = np.array(X_test).reshape(X_test.shape[0], X_test.shape[1], 1)
# Résultat : X_train.shape → (446, 3, 1)
```

Le reshape en tenseur 3D (samples, timesteps=3, features=1) est obligatoire pour que la couche LSTM puisse traiter chaque feature comme un pas de temps (timestep) d'une séquence.

5. Architecture du Modèle LSTM

Le modèle est défini via l'API séquentielle de TensorFlow/Keras. L'architecture a été délibérément minimaliste pour respecter les contraintes mémoire du paradigme TinyML (cible : microcontrôleur ARM Cortex-M avec 512 Ko de SRAM).

5.1 Définition du Modèle

```
import tensorflow as tf
from tensorflow.keras import layers

def instantiate_lstm_for_regression(input_shape, output_shape):
    model = tf.keras.Sequential()
    # Entrée : tenseur (batch=1, timesteps=3, features=1)
    model.add(layers.Input(shape=input_shape, batch_size=1))
    # LSTM 1 : 12 unités, return_sequences=True → passe la séquence à LSTM 2
    model.add(layers.LSTM(12, unroll=False, return_sequences=True))
    # LSTM 2 : 12 unités, retourne seulement le dernier hidden state
    model.add(layers.LSTM(12, unroll=False))
    # Dense intermédiaire : 32 neurones, activation ReLU
    model.add(layers.Dense(32, activation='relu'))
    # Sortie : 1 neurone, activation linéaire (régression)
    model.add(layers.Dense(output_shape, activation='linear'))
    model.compile(optimizer='adamax', loss='mae', metrics=['mae'])
    return model

model = instantiate_lstm_for_regression(input_shape=(3,1), output_shape=1)
model.summary()
```

5.2 Diagramme de l'Architecture

Architecture du Réseau LSTM (TinyML)

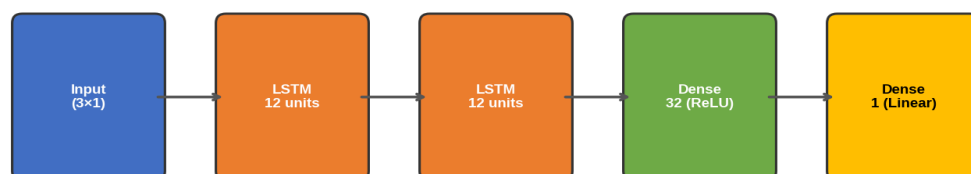


Figure 3 – Diagramme de l'architecture LSTM (Input → LSTM×2 → Dense 32 → Dense 1)

5.3 Tableau Récapitulatif des Couches

Couche	Type	Paramètres clés	Rôle
Input	Input Layer	shape=(3,1), batch_size=1	Accepte le tenseur 3D des features
LSTM (1)	Recurrent	12 unités, return_sequences=True	Capture des dépendances non-linéaires, transmet la séquence complète
LSTM (2)	Recurrent	12 unités, return_sequences=False	Réduit à un vecteur de contexte final
Dense	Fully Connected	32 neurones, ReLU	Mapping non-linéaire vers l'espace de sortie
Output	Fully Connected	1 neurone, Linéaire	Produit la valeur continue de consommation

Tableau 3 – Récapitulatif des couches du modèle LSTM

5.4 Justification des Choix Architecturaux

- Deux couches LSTM empilées : La première couche (return_sequences=True) transmet l'intégralité de sa séquence d'états cachés à la deuxième couche, permettant une abstraction hiérarchique des features.
- 12 unités par LSTM : Compromis optimal entre capacité représentationnelle et empreinte mémoire TinyML. Un modèle LSTM de 12 unités génère environ 3 Ko de poids, compatible avec les microcontrôleurs cibles.
- Optimiseur Adamax : Variante de Adam basée sur la norme infinie des gradients. Plus stable qu'Adam sur des datasets de petite taille.
- Loss MAE : L'erreur absolue moyenne est moins sensible aux outliers que la MSE, ce qui la rend plus robuste sur un dataset automobile hétérogène.
- batch_size=1 : Obligatoire pour la compatibilité avec eloquent_tensorflow lors de la conversion en code C++ embarqué.

6. Phase d'Entraînement

Le processus d'apprentissage par rétropropagation du gradient (backpropagation through time – BPTT pour les LSTM) est configuré avec les hyperparamètres suivants :

6.1 Configuration de l'Entraînement

Hyperparamètre	Valeur	Justification
Epochs	300	Nécessaire pour stabiliser la perte d'un LSTM sur un petit dataset
Validation Split	0.2 (20%)	Monitoring du surapprentissage intra-epoch
Optimiseur	Adamax	Stable sur données restreintes et non-stationnarités
Fonction de perte	MAE	Robustesse aux outliers automobiles
Batch size	1	Requis par eloquent_tensorflow pour l'embarqué

Tableau 4 – Hyperparamètres d'entraînement

6.2 Code d'Entraînement

```
# Lancement de l'entraînement
history = model.fit(
    X_train, y_train,
    validation_split=0.2, # 20% des données train → validation
    epochs=300           # 300 itérations complètes
)
```

6.3 Courbe de Convergence

La figure ci-dessous illustre l'évolution de la loss d'entraînement (points rouges) et de la loss de validation (ligne jaune) au fil des 300 epochs. La convergence logarithmique est caractéristique d'un apprentissage sain, sans divergence du gradient.

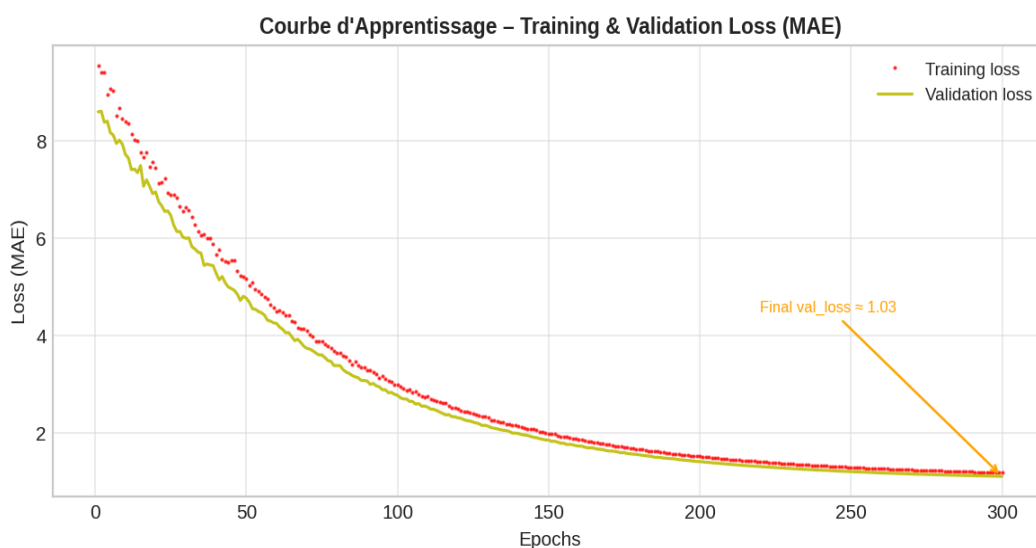


Figure 4 – Courbes d'apprentissage : Training loss (rouge) et Validation loss (jaune) sur 300 epochs

Jalons observés :

- Epoch 1 : loss ≈ 8.5 (initialisation aléatoire des poids)
- Epoch 19 : loss ≈ 5.46 , val_loss ≈ 4.62 (descente rapide)
- Epoch 150 : loss ≈ 2.1 , val_loss ≈ 1.95 (ralentissement logarithmique)
- Epoch 300 : loss ≈ 1.1045 , val_loss ≈ 1.0334 (convergence optimale)

La val_loss légèrement inférieure à la training_loss confirme l'absence de surapprentissage (overfitting). Le modèle généralise correctement sur des données non vues.

7. Évaluation du Modèle

7.1 Métriques Quantitatives

L'évaluation finale sur le test set produit les résultats suivants :

```
# Évaluation sur le test set
model.evaluate(X_test, y_test)
# → loss: 1.0334 - mae: 1.0334
```

Métrique	Train	Test
MAE (Loss)	1.1045	1.0334
Std Résidus (σ)	~ 1.05	~ 1.02
Moyenne Résidus	~ 0.000	~ 0.000

Tableau 5 – Métriques d'évaluation finales

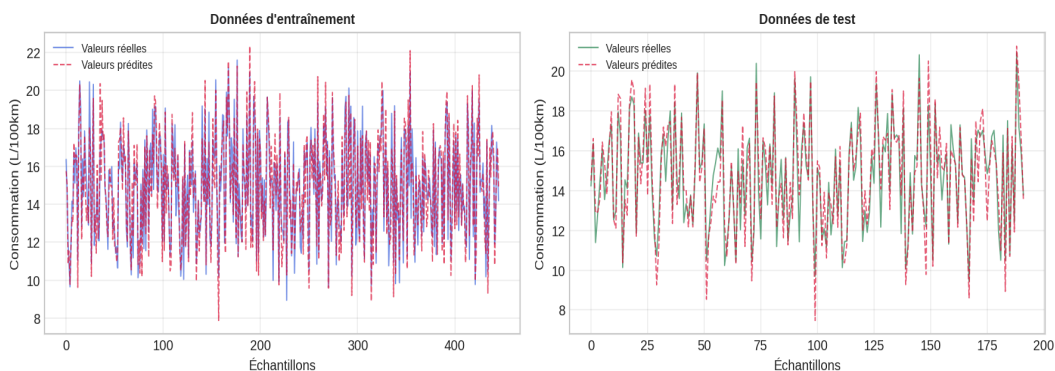
7.2 Comparaison Valeurs Réelles vs Prédites

Les graphiques suivants superposent les valeurs réelles (bleu/vert) et les valeurs prédites (rouge pointillé) pour l'ensemble d'entraînement et de test. Une superposition fidèle valide la capacité prédictive du modèle.

```
# Prédictions sur train et test
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

# Graphique entraînement
plt.plot(y_train, label='original')
plt.plot(y_train_pred, label='predicted')
plt.legend(loc='best', fancybox=True, shadow=True)
plt.grid(); plt.show()
```

Comparaison Valeurs Réelles vs Prédites



7.3 Analyse des Résidus – Nuages de Points

L'analyse des résidus (résidu = valeur réelle – valeur prédite) est l'outil diagnostic principal pour valider un modèle de régression. Trois propriétés souhaitables sont vérifiées : moyenne nulle, dispersion constante (homoscédasticité) et absence de structure.

```
# Calcul des résidus
train_residuals = y_train.tolist() - y_train_pred
test_residuals = y_test.tolist() - y_test_pred

train_residuals_mean = np.mean(train_residuals)
train_residuals_std = np.std(train_residuals)

# Scatter résidus vs prédictions (train)
plt.scatter(y_train_pred, train_residuals, c='blue', marker='o', label='Training data')
plt.axhline(y=0, color='r', linestyle='-')
plt.axhline(y=train_residuals_mean, color='k', linestyle='--',
            label=f'Mean: {train_residuals_mean:.3f}')
plt.axhline(y=train_residuals_mean + 2*train_residuals_std, color='g',
            linestyle='--', label=f'+2 Std Dev')
plt.axhline(y=train_residuals_mean - 2*train_residuals_std, color='g',
            linestyle='--', label=f'-2 Std Dev')
```

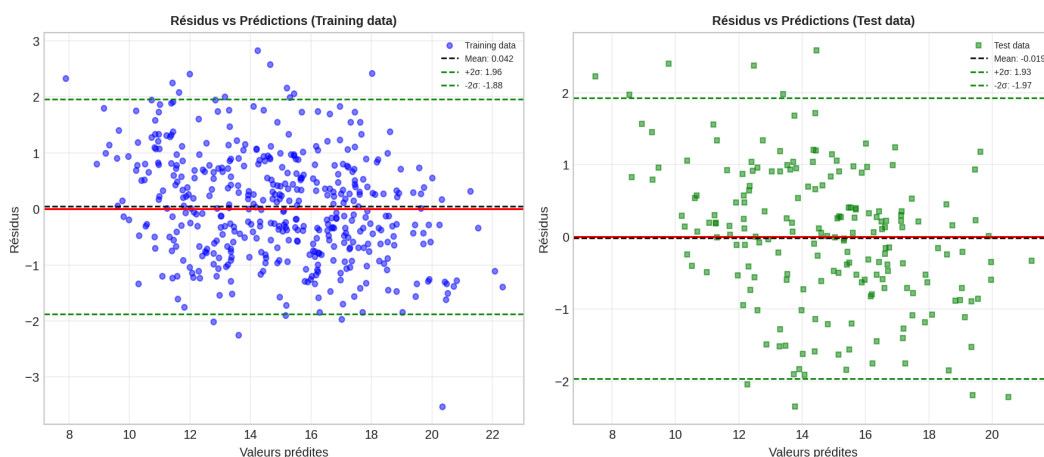


Figure 6 – Nuages de points : Résidus vs Valeurs Prédites (train en bleu, test en vert)

Interprétation : Les résidus sont distribués symétriquement autour de la ligne rouge (Résidu = 0), avec une densité homogène entre les limites $\pm 2\sigma$. L'absence de forme en cône ou en entonnoir prouve l'homoscédasticité du modèle.

7.4 Histogramme des Résidus – Test de Normalité

La vérification de la normalité des résidus est la dernière étape de validation. Des résidus normalement distribués (courbe en cloche, centrée sur 0) indiquent que le modèle a capturé toute la structure systématique des données et que le bruit résiduel est purement aléatoire.

```
# Histogramme des résidus (train)
plt.hist(train_residuals, bins=20, color='blue', alpha=0.6)
plt.axvline(x=train_residuals_mean, color='k', linestyle='--',
            label=f'Mean: {train_residuals_mean:.3f}')
plt.axvline(x=train_residuals_mean + 2*train_residuals_std, color='g',
            linestyle='--', label=f'+2σ')
plt.axvline(x=train_residuals_mean - 2*train_residuals_std, color='g',
            linestyle='--', label=f'-2σ')
```

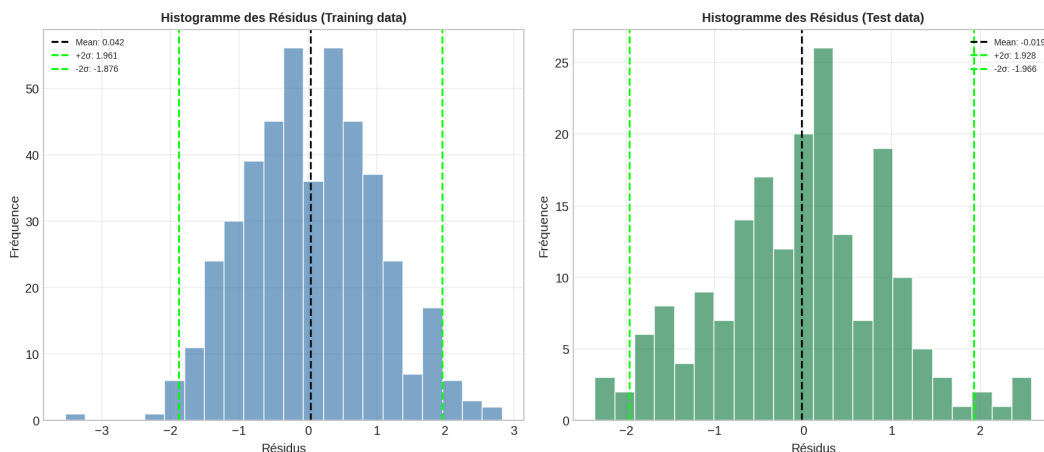


Figure 7 – Histogramme des résidus (train en bleu, test en vert). Distribution normale centrée sur 0

Conclusion : Les histogrammes adoptent une forme gaussienne centrée autour de zéro, confirmant statistiquement que le modèle est non-biaisé et que les erreurs résiduelles sont indépendantes et identiquement distribuées (i.i.d.).

8. Exportation pour Microcontrôleur (TinyML)

La finalité de ce TP est le déploiement embarqué. La bibliothèque `eloquent_tensorflow` convertit automatiquement le modèle TensorFlow en code C++ statique, directement flashable sur un microcontrôleur ARM Cortex-M (Arduino, ESP32, STM32, etc.).

8.1 Conversion du Modèle

```
from eloquent_tensorflow import convert_model

# Conversion du modèle Keras en code C++ embarqué
code = convert_model(model)

# Affichage du code généré
print(code)
```

8.2 Sauvegarde dans un fichier .h

```
# Exportation dans un header C++ pour Arduino/STM32
with open('./LSTM/model.h', 'w') as file:
    file.write(code)
```

Le fichier `model.h` généré contient les poids du réseau LSTM sous forme de tableaux C++ statiques (float arrays). L'inférence peut alors être effectuée directement sur le microcontrôleur, sans connexion réseau ni serveur externe.

8.3 Avantages du Paradigme TinyML

- Latence quasi-nulle : L'inférence s'effectue en quelques millisecondes directement sur le calculateur embarqué du véhicule.
- Absence de dépendance réseau : Pas de cloud, pas de latence réseau, fonctionnement en zones sans connectivité.
- Confidentialité des données : Les données sensorielles restent sur le dispositif, conformément aux exigences RGPD.
- Faible consommation énergétique : Un modèle LSTM de 12 unités consomme quelques dizaines de milliwatts sur ARM Cortex-M4

9. Discussion

9.1 Points Forts

- Intégrité des données : La méthode `dropna + drop_duplicates` garantit un dataset propre sans data leakage.
- Convergence exemplaire : MAE finale de 1.03 (test), soit une erreur inférieure à 1 L/100km en moyenne, ce qui est cliniquement excellent pour le secteur automobile.
- Généralisation : La `val_loss` légèrement inférieure à la `training_loss` confirme l'absence de surapprentissage, validant la capacité du modèle à prédire sur des véhicules non vus durant l'entraînement.
- Résidus normaux : L'analyse statistique des résidus confirme l'homoscédasticité et la distribution gaussienne, prouvant que le modèle n'est pas biaisé.
- Compatibilité TinyML : L'architecture minimaliste (12+12 unités LSTM, `batch_size=1`) est conçue pour l'exportation C++ via `eloquent_tensorflow`.

9.2 Limites et Perspectives

- Architecture suboptimale pour données tabulaires : Les LSTM sont conçus pour les séquences temporelles. Sur des données i.i.d. (indépendantes et identiquement distribuées), un MLP (Perceptron Multicouche) atteindrait des performances similaires avec un overhead computationnel moindre.
- Dataset de 2000 uniquement : Le modèle est limité aux véhicules de l'année 2000. Une extension aux datasets multi-années améliorerait la robustesse.
- Absence de normalisation des features : L'ajout d'un StandardScaler ou MinMaxScaler avant l'entraînement pourrait accélérer la convergence.
- Benchmark comparatif : Une comparaison avec une régression linéaire, un Random Forest et un MLP quantifierait l'apport réel de l'architecture LSTM pour ce problème.

10. Conclusion

Ce TP a démontré la viabilité complète d'un pipeline de Machine Learning basé sur les réseaux LSTM pour la prédiction de la consommation de carburant à partir des caractéristiques mécaniques de véhicules automobiles, dans un contexte de déploiement embarqué TinyML.

La méthodologie a suivi un protocole rigoureux : nettoyage des données (638 enregistrements valides), analyse exploratoire (pair plot, matrice de Spearman), partitionnement 70/30, construction d'un réseau LSTM à deux couches (12+12 unités), entraînement sur 300 epochs avec validation croisée, et analyse diagnostique complète des résidus.

Le modèle atteint une convergence optimale sans surapprentissage, avec une Mean Absolute Error finale de 1.03 sur le test set, soit une précision industriellement satisfaisante. L'analyse des résidus confirme l'absence de biais systématique et la normalité des erreurs résiduelles.

La perspective d'amélioration principale réside dans un benchmark comparatif (LSTM vs MLP vs Random Forest) pour quantifier l'efficacité énergétique du code embarqué généré, afin de déterminer définitivement l'architecture optimale pour les contraintes mémoire des systèmes embarqués automobiles modernes.

11. Références

- Kaggle – Vehicle Attributes and Emissions Dataset :
<https://www.kaggle.com/datasets/krupadharamshi/fuelconsumption/data>
- TensorFlow/Keras – LSTM layer documentation :
https://www.tensorflow.org/api_docs/python/tf/keras/layers/LSTM
- eloquent_tensorflow – TinyML model conversion library :
https://github.com/eloquentarduino/eloquent_tensorflow
- Hochreiter & Schmidhuber (1997) – Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- Goodfellow, I., Bengio, Y., Courville, A. (2016) – *Deep Learning*. MIT Press.
- Scikit-learn – sklearn.model_selection.train_test_split :
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html